

TeachMate: A Teaching Assistant using Small Language Models for Education in Remote Areas

Erol John D'Silva

*Department of Electrical and Electronics Engineering
Amrita School of Engineering, Bengaluru
Amrita Vishwa Vidyapeetham, India
bl.en.p2ebs24003@bl.students.amrita.edu*

K. Sireesha

*Department of Electrical and Electronics Engineering
Amrita School of Engineering, Bengaluru
Amrita Vishwa Vidyapeetham, India
k_sireesha@blr.amrita.edu*

Abstract—This work shows how we built a dataset from the Karnataka State Board curriculum for Class 6 Mathematics, designed to train a small language model to behave as a teaching assistant that is able to support students by enabling question answering, notes generation and provide feedback to the teacher by monitoring student progress, while being grounded in the required curriculum. To understand how existing models perform on such tasks, we perform a comparative study between a few popular language models and the results show that fine-tuned models are more likely to answer domain-specific questions correctly, thus setting a foundation for small language models to be used for regional education. We also find the best performing model architecture for resource-constrained devices, establishing a framework for AI-driven learning with low resource consumption. These objectives align with Sustainable Development Goal 4 (SDG 4) with respect to Quality Education.

Index Terms—Small Language Model, Domain-specific, AI-driven, SDG 4, Quality Education.

I. INTRODUCTION

Artificial Intelligence (AI) is widely used today as a tool to access information. The ability of AI models like Large Language Models (LLMs) to have a human-like conversation, makes the experience personalised. For this reason, most students prefer learning new concepts via interaction with LLMs. The drawback of these models is that they require large amount of resources in the form of computational power and continuous internet connectivity. These models also have a dependency on cloud infrastructure for storage and compute. For these reasons, LLMs can not be used in areas with little to no network connectivity. To host LLMs locally, the cost of hardware infrastructure is quite high. In the context of education, LLMs are able to provide an interactive style of explanation of various concepts, however, they are prone to a phenomenon called hallucination. This scenario occurs when the language model is stuck in a loop of similar "thoughts", resulting in the model fabricating the output. This means that sometimes the output of the model may be fictional. For a teaching assistant tool, it is important that the information provided is accurate and of the highest quality. In many parts of India, there is a shortage of teaching professionals in remote areas, affecting the availability of quality education to students in these parts of the country. To overcome these challenges, we propose a Small Language Model (SLM), which is specific to

the Karnataka State Board curriculum and fine-tuned for 6th Grade Mathematics. This solution addresses the mentioned challenges by serving as a virtual teaching assistant, with the goal to provide a solution that can be scaled in order to improve access to quality education to all students in India. Our objectives align with the United Nations (UN) Sustainable Development Goal 4 (SDG 4) - which is Quality Education and opportunity for all, via equitable and easy access to education across the country.

II. RELATED WORK

The rapid advancement of Artificial Intelligence has significantly influenced the development of educational technology. Large Language Models (LLMs) have demonstrated strong capabilities in reasoning, summarization and contextual understanding. This makes them an attractive solution for applications that involve learning. However, the high computational and memory requirements limit their ability to function on devices with constrained resources, such as embedded devices. This is the motivation for research on Small Language Models (SLMs) that are optimized for deployment on devices with limited memory [1], [2].

Research has explored application of natural language processing (NLP) for education tasks that include regional languages. Some use cases such as classification of Kannada and Telugu news articles [3], emotion prediction via extended multilingual text analysis [4] and domain-specific embeddings for Bengali corpus [5] show the importance of data being aligned to the particular use case. Educational assistants have been proposed using BERT architecture [6] along with personalized education with adaptive learning systems [7]. Study on the trade-off between accuracy and computational efficiency was shown in [8], thus promoting the need for lightweight language model architectures in the education domain.

The transition from recurrent neural network (RNN) approaches to attention-based architectures marked a major shift in NLP. Models that use the transformer architecture show efficient parallelization and improved contextual understanding, and form the foundation of most modern language models [9]. Unified text-to-text frameworks further simplify task formulation and model reuse across diverse NLP problem areas, with

multilingual extensions of models enabling broader coverage for regional languages [1], [10].

A key challenge in applications for the education space is adapting general models to domain specific use case. Models trained on general purpose data, tend to underperform in structured environments, where the models require domain alignment. Prior work has shown training models on domain-specific data, improves performance in question answering and reasoning tasks [11], [12]. Studies on bilingual and multilingual datasets further demonstrate that inclusion of regional language increases learner comprehension, particularly in STEM education [13]. Additionally, AI systems should account for limitations in low-resource hardware environments, as it directly influences model selection and deployment strategies [14].

Complete fine-tuning of language models from scratch is computationally expensive and economically impractical for most applications. Parameter-Efficient Fine-Tuning (PEFT) techniques address this limitation by updating only a small number of selected parameters, while keeping the rest of the model frozen [15], [16]. Among these methods, Low Rank Adaptation (LoRA) has emerged as the most effective, with strong task adaptation with significantly reduced training cost. Comparative analysis of PEFT techniques indicate that LoRA offers a favorable balance between efficiency and performance, especially for models required to be deployed on embedded devices [17]. Alternative approaches such as prefix tuning introduce continuous prompts into the attention mechanism but prove to be less effective for tasks that involve complex numerical and logical reasoning [18].

Recent years have seen many compact and capable model architectures suitable for edge deployment. Instruction-tuned models improve task generalization and robustness across educational use cases [2]. Lightweight multilingual models provide support for regional languages, while architectures tuned on domain-specific data demonstrate improved numerical reasoning and structured response generation [10], [12]. These findings indicate that model choice must balance reasoning capability, language support and computational efficiency while designing on-device educational assistants.

III. PROBLEM FORMULATION

The primary objective of this project is to address the unavailability of quality education in parts of India by providing a virtual teaching assistant in the form of an offline application.

We aim to create the entire data pipeline which will outline the steps taken to create the custom dataset. This will provide a scalable pipeline that can be reused to increase the size of the dataset in the future. The dataset will be created in such a way that it can be used to finetune small language models. Multiple models will be evaluated and compared to find the best suitable model for our use case. This model will then be trained on our dataset using parameter efficient fine-tuning (PEFT) techniques. The trained model will be evaluated and selected to be used for our solution. We will design and build a system that can perform the functionality required from

a teaching assistant tool. The system will be developed in two parts – the software part includes the user interface and software to run the application, and the hardware part includes the integration of required hardware to support the software functionality. The objectives mentioned align with our goal, which from a social standpoint, is to provide a tool that allows access to quality education for all students across India. From a research point of view, we plan to explore the domain of language models on devices with low compute for the use cases in domain of education.

IV. METHODOLOGY

The work done in this study follows a systematic pipeline which consists of dataset preparation and processing, model selection, model fine-tuning and model benchmarking and evaluation. These sections will be explained in the course of this paper.

A. Dataset Pipeline

The data pipeline consists of the following sections as described below. These are made to be modular such that this pipeline can be reused to extend this dataset to include other aspects of the curriculum, including other subjects.

1) *Data Acquisition*: For the collection of required data we had to first find the relevant source. The official website of the Karnataka State Board was found to have the digital copies of textbooks according to the established curriculum. This was a good starting point for us to begin our data collection. The digital textbooks were available in downloadable PDF format.

2) *Data Processing*: Once the PDF files were downloaded and available in local storage, we used the Python programming language for scripting. These scripts can perform functions like converting the data into text format, by parsing the files via available python libraries. The data generated was found to have a lot of garbage values, and hence required us to write a script to clean the text. After the removal of unwanted characters and symbols, we had the clean text data that can further be processed.

3) *Data Structuring*: The data is now divided into groups based on the type of content. After manual inspection of the textbook and its contents, we observe three distinct types of data blocks. The explanation block contains the written explanation of certain concepts, for example a section titled “What is Mathematics?”. The text block that follows this title is considered by us to be an explanation block and is grouped accordingly in the dataset. The example block contains the steps to solve questions and has solved examples from the textbook. The final section is the exercise section which contains exercise questions from the textbook.

4) *Data Formatting*: The data is saved as two different types – QnA pairs and Tokenized data points. The QnA pairs will be used to fine-tune models for the SLM that will be used for the chatbot functionality. The tokenized data points will be used to fine-tune the model that will be used to summarize text. The data is saved and stored in host’s local storage as a .json file. The final size of the dataset is 10400 QnA pairs

TABLE I
EVALUATION METRICS

Metric	Purpose	Formula
F1 Score	Precision-Recall balance	$\frac{2PR}{P+R}$
Numeric Accuracy	Exact numeric match	$\frac{1}{N} \sum (y = y')$
MAE	Average numeric error	$\frac{1}{N} \sum y - y' $
ROUGE-L	LCS-based similarity	$\frac{(1+\beta^2)PLCS \cdot RLCS}{\beta^2 PLCS + RLCS}$

TABLE II
SYSTEM PERFORMANCE METRICS

Metric	Purpose	Formula
Latency	Inference delay	$\frac{1}{N} \sum (t_{end} - t_{start})$
Model Size	Deployment feasibility	Params $\times$ Precision
RAM Usage	Peak memory usage	$\max(RAM_{end} - RAM_{start})$

with a percentage split of 70-20-10 for training, validation and testing.

B. Model Selection

Before we can fine-tune models on our dataset, we must select and benchmark a few models that we can use as potential candidates for our solution. To do this we run the selected models on our host device and record the performance metrics. This serves as a baseline for comparison with models after fine-tuning. As observed in our literature survey, there are many language models that currently exist. To appropriately select models for our use case, we have used some selection criteria:

- High accuracy on instruction style of data input.
- Low latency to process tokens
- Small model size with lesser number of parameters
- Works well with Math questions and answers

Based on model architecture that supports the above selection criteria, we selected the following families of models – Phi, Qwen and Llama. These architectures are commonly used for interactive tasks such as questions and answers. To further select the models from sub-classes of these families, we relied on literature results and our selection criteria to choose the models that best fit our requirements. The models are selected from literature based on model architecture, advantages and number of parameters, while keeping in mind our selection criteria.

C. Evaluation Parameters

The selected models must be evaluated using certain performance metrics in order to compare and select the best performing model for our use case. The evaluation pipeline shown in Fig 3 shows the steps followed for model evaluation. To measure the mathematical reasoning ability of the models, we use a few parameters for evaluation of the baseline models. These parameters are shown in Table I, along with the reason for their selection and their mathematical formulae. Since our goal is to deploy the model on memory constrained devices, we have to evaluate the model on metrics with regards to hardware performance using the metrics shown in Table II.

D. Fine-tuning Selected Models

The fine-tuning methodology adopted in this work is Low-Rank Adaptation (LoRA), which is well supported in the literature for adapting large and small language models using limited training data while updating only a small subset of parameters. This approach is particularly advantageous for language models, where full fine-tuning is computationally expensive and economically impractical due to the large number of parameters involved.

Using LoRA, the base model weights are kept frozen, and trainable low-rank matrices are introduced into selected attention projection layers. During training, only these low-rank matrices are updated, enabling efficient task adaptation without modifying the full model. In this work, LoRA is applied to the attention projection layers of the Qwen model, allowing effective specialization to the educational domain under constrained compute and memory budgets.

The hyperparameters used for LoRA-based fine-tuning, including rank, target modules, and training configuration, are summarized in Table III. Details of the dataset employed for fine-tuning, including the number of samples and the train-validation-test split, are described in the dataset preparation section.

V. EXPERIMENTAL SETUP

Three instruction-tuned language models of varying parameter scales are evaluated to study the trade-off between performance and deployability on resource-constrained systems. All models are fine-tuned using parameter-efficient techniques to ensure feasibility under limited memory and compute budgets.

Fine-tuning is performed using Low-Rank Adaptation (LoRA) applied to the attention projection layers, combined with 4-bit quantization to reduce memory footprint. Training is conducted with a batch size of 2 and gradient accumulation of 4 steps, resulting in an effective batch size of 8. The learning rate is fixed at 0.0002, training is run for 3 epochs, and the maximum input sequence length is limited to 256 tokens, as seen in Table III. Models are evaluated on a numerically grounded text generation task requiring both semantic correctness and precise numerical outputs. Performance is measured using F1 score, numeric accuracy, mean absolute error (MAE), and ROUGE-L. Inference latency, model size, and peak RAM usage are additionally recorded to assess deployment feasibility on embedded hardware.

The models are trained using Google Colab T4 GPU. Each model is evaluated before and after optimization under identical conditions. Reported results are averaged across multiple runs to reduce system-level variance and ensure fair comparison.

VI. RESULTS AND ANALYSIS

After the models were fine-tuned, they were once again evaluated based on the performance metrics discussed in the previous sections. The results of which will be discussed in this section. The experimental results before finetuning are shown in Fig 1. We observe that the Phi model is performing

TABLE III
FINE-TUNING CONFIGURATION

Parameter	Value
Quantization	4-bit
LoRA Rank (r)	16
LoRA Modules	q, k, v proj
Learning Rate	2×10^{-4}
Batch Size	2
Grad. Accumulation	4
Epochs	3
Max Seq. Length	256

TABLE IV
RESULTS BEFORE FINE-TUNING

Model	F1	Acc.	MAE	R-L
Phi-3-mini-4B-Instruct	0.314	0.90	70.10	0.915
Qwen2.5-3B-instruct	0.392	0.61	110.60	0.505
TinyLlama-1.1B	0.377	0.26	505.06	0.260

quite well across all the performance metrics. The values obtained in Table IV show us the calculated values of all the models before finetuning. We can observe from these results that the Qwen model performs the best after being fine-tuned on our custom Math dataset. Table IV summarizes the aggregated model scores across all evaluated metrics. As shown in Table IV, Qwen2.5-3B-Instruct achieves the highest overall score (0.718), exceeding Phi-3-mini-4B-Instruct (0.403) and TinyLlama-1.1B (0.425). This result indicates that, when considering both task-level performance and system-level constraints jointly, Qwen2.5-3B-Instruct provides the most favorable balance among the evaluated models.

Tables VII and VI present model performance before and after optimization, respectively. Comparing Table VII to Table VI, all models exhibit improvements in F1 score and reductions in MAE following fine-tuning and quantization. Specifically, Qwen2.5-3B-Instruct improves from an F1 score of 0.392 to 0.432 and reduces MAE from 110.60 to 9.91, while TinyLlama-1.1B improves from 0.377 to 0.416 in F1 and reduces MAE from 505.06 to 67.31. Phi-3-mini-4B-Instruct also shows a reduction in MAE from 70.10 to 43.71, though its gains in F1 are comparatively smaller.

Table V presents the percentage improvements which show how much better the system performs compared to its baseline performance. According to Table V, Qwen2.5-3B-Instruct

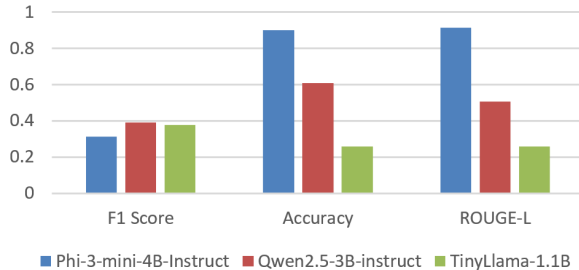


Fig. 1. Evaluation Before Fine-tuning

TABLE V
RESULTS AFTER FINE-TUNING

Model	F1	Acc.	MAE	R-L
Phi-3-mini-4B-Instruct	0.383	0.40	43.71	0.419
Qwen2.5-3B-instruct	0.432	0.93	9.91	0.513
TinyLlama-1.1B	0.416	0.84	67.31	0.444

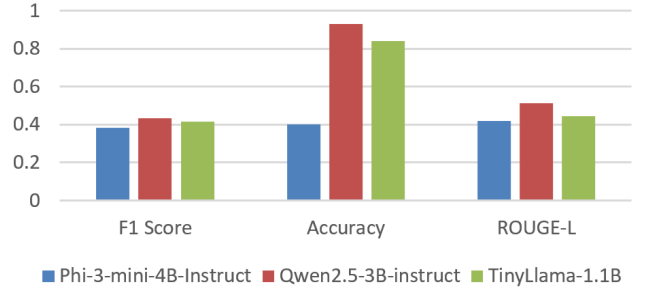


Fig. 2. Evaluation After Fine-tuning

delivers the highest MAE decrease of 83.55% while keeping a small ROUGE-L improvement of 0.78%. The system produces better numerical results, yet it stays with its original output structure. The fine-tuning process leads to absolute error values which become higher for smaller models than the Qwen2.5-3B-Instruct model. The model Phi-3-mini-4B-Instruct experiences a major decline in performance because its accuracy drops by 38.00%, and its ROUGE-L score decreases by 37.18%. The models which use LoRA for fine-tuning show differences in their ability to keep semantic meaning after being fine-tuned on domain-specific data.

The tables from IV to VII demonstrate that mid-sized models show the best performance when working with limited resources. The data shows that small models reach substantial percentage growth according to Table V yet their actual performance levels after optimization stay at low levels based on Table VI. The Phi-3-mini-4B-Instruct model shows unstable behavior through different performance indicators after optimization according to Tables V and VI. The models are given a score which is calculated as:

$$\text{Score} = 0.25 (F_1 + \text{Acc} + (1 - \text{NMAE}) + \text{R-L})$$

The research findings establish Qwen2.5-3B-Instruct as the best model for deployment on hardware machines which have limited computing resources, and finds its main use when used for specialized training of domain-specific tasks.

VII. DISCUSSION

The study shows that parameter-efficient fine-tuning together with strong quantization methods enhance numerical reasoning abilities of systems which operate under limited resource availability but the success of this method depends on which model gets selected. The optimization process showed continuous progress which proved that low-rank adaptation works effectively for fixing task-specific issues while avoiding complete model retraining and maintaining 4-bit precision levels.

TABLE VI
PERCENTAGE GAIN OVER BASELINE

Model	F1 (%)	Acc. (%)	MAE (%)	R-L (%)
Phi-3-mini-4B-Instruct	9.89	-38.00	-23.18	-37.18
Qwen2.5-3B-instruct	4.85	20.77	-83.55	0.78
TinyLlama-1.1B	4.91	52.72	-76.48	26.13

TABLE VII
OVERALL MODEL SCORE

Model	Score
Phi-3-mini-4B-Instruct	0.403
Qwen2.5-3B-instruct	0.718
TinyLlama-1.1B	0.425

The evaluated models display different patterns which show that larger models produce better results but they create more instability in the optimization process. The Qwen2.5-3B-Instruct model with its mid-sized architecture shows the most consistent improvement through optimization because it produces both accurate numbers and well-structured output. The models which operate within this parameter range demonstrate their ability to handle quantization noise through their existing representational abilities while preserving their learning capacities with LoRA. The smallest model shows big percentage improvements yet it achieves low absolute accuracy which proves that parameter efficiency does not fix the problem of having less model capacity for tasks that need numerical grounding.

The degradation observed in certain metrics for Phi-3-mini-4B-Instruct points to a sensitivity to optimization under constrained precision. The model possesses more parameters than TinyLlama-1.1B but it fails to maintain stable numerical accuracy and structural similarity patterns which indicates that its design elements and initial training process determine its ability to handle reduced bit precision during fine-tuning. The deployment process needs more than model size information because this information does not predict how well models function with restricted memory and limited precision resources. This can be attributed to the model being trained on general data, which limits the ability of the model to understand text structure associated with mathematics. Furthermore the Phi-3 family of models has proven to show deteriorating accuracy with reduction in model size.

The deployment perspective reveals that organizations need to base their model selection process on absolute performance achievements rather than relative improvement numbers when choosing embedded and edge system models. The error levels after optimization stay too high for numerical reliability tasks when using smaller models which achieve better percentage improvements. Models which demonstrate consistent metric progress become more reliable for predicting their performance when operating in actual environments.

The research findings demonstrate that models need to be selected carefully along with their optimization methods to achieve success with on-device training and inference systems. The deployment of mid-sized instruction-tuned models be-

comes possible because these models strike a balance between running efficiently and maintaining strong numerical stability which makes them perfect for restricted operational spaces. Some cases where fine-tuning fails to be effective is for multi-step Mathematical problems. However, the Qwen model still performs significantly better than other models in this study, showing the importance of instruct models. This further solidifies the argument that training Qwen model on domain specific data leads to better performance than training on general data.

VIII. LIMITATIONS

The current implementation of TeachMate is limited to a single subject domain - 6th Grade Mathematics and a restricted curriculum scope. Although parameter-efficient fine-tuning improves numerical reasoning, the model may still generate incorrect explanations for complex multi-step problems. Evaluations were conducted in controlled experimental settings and do not fully capture real classroom usage or student interaction patterns. Additionally, while the system is designed for low-resource deployment, extensive validation on ultra-low-power embedded hardware remains future work.

IX. CONCLUSION AND FUTURE WORK

This work presents the development of a virtual teaching assistant tool capable of answering curriculum related questions and summarizing study text. We have shown how SLMs can be used in resource constrained environments by the use of training and fine-tuning techniques. We have shown model selection across multiple state of the art models from literature. After baseline evaluation of the selected models, we fine-tuned them on our custom dataset using low rank adaptation as the fine-tuning methodology. A comparative analysis was done to show the performance of models before and after fine-tuning and the percentage gain or loss in performance. The models were scored based on their performance and the Qwen model came out the winner and was thus used in our application. We also validated the chatbot and summarizer outputs and created a mock user interface to show what the application will look like. The future scope of the project lies in improving model performance, feature addition and hardware build and deployment, such that the application can run on the selected hardware. Training models on regional data is possible using this pipeline, and the performance of Qwen model on regional language data like Kannada can be explored in future work. The modular design of our pipeline allows it to be reused to create datasets for study material in different languages allowing the scope for SLMs in Education to reinforced further.

REFERENCES

- [1] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [2] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, Y. Li *et al.*, "Scaling Instruction-Finetuned Language Models," *Journal of Machine Learning Research*, vol. 25, no. 70, pp. 1–53, 2024.

- [3] D. Gupta, K. Thejaa, A. R. Nair, and P. Katti, "Automated regional language news article classification system for Kannada and Telugu," in *Proc. Fifth Int. Conf. Advances in Electrical, Computing, Communication and Sustainable Technologies (ICAECT)*, IEEE, 2025, pp. 1–6.
- [4] J. S. Reddy, D. A. Surat, P. Shyamala, and S. Syama, "Emotion prediction from text and multilingual voice inputs," in *Proc. 8th Int. Conf. Electronics, Communication and Aerospace Technology (ICECA)*, IEEE, 2024, pp. 850–855.
- [5] A. Bandyopadhyay and J. Nair, "Word embedding for Bengali language using domain-related corpus," in *Proc. Int. Conf. Inventive Computation Technologies (ICICT)*, IEEE, 2023, pp. 896–901.
- [6] V. S. Lalapura, V. R. Bhimavarapu, J. Amudha, and H. S. Satheesh, "A systematic evaluation of recurrent neural network models for edge intelligence and human activity recognition applications," *Algorithms*, vol. 17, no. 3, p. 104, 2024.
- [7] C. Sriharsha, S. R. K. Rithwik, K. P. Prahlad, and L. S. Nair, "Intelligent learning assistant using BERT and LSTM," in *Proc. 12th Int. Conf. Computing Communication and Networking Technologies (ICCCNT)*, IEEE, 2021, pp. 1–6.
- [8] A. S. Kuttattu, G. S. Gokul, H. Prasad, J. Murali, and L. S. Nair, "Analysing the learning style of an individual and suggesting field of study using machine learning techniques," in *Proc. Int. Conf. Communication and Electronics Systems (ICCES)*, IEEE, 2019, pp. 1671–1675.
- [9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [10] D. Kakwani, A. Kunchukuttan, S. Golla, G. N. C., A. Bhattacharyya, M. M. Khapra, and P. Kumar, "IndicNLP Suite: Monolingual Corpora, Evaluation Benchmarks, and Pre-trained Multilingual Language Models for Indian Languages," in *Findings of the Association for Computational Linguistics: EMNLP*, 2020, pp. 4948–4961.
- [11] A. Agarwal, N. Sachdeva, R. K. Yadav, V. Udandaraao, V. Mittal, A. Gupta, and A. Mathur, "EDUQA: Educational Domain Question Answering System Using Conceptual Network Mapping," in *Proc. IEEE Int. Conf. on Acoustics, Speech and Signal Processing (ICASSP)*, 2019, pp. 8137–8141.
- [12] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan *et al.*, "Qwen Technical Report," *arXiv preprint arXiv:2309.16609*, 2023.
- [13] N. Akramova, S. Akhmedov, M. Mukhtorova, F. Nasretdinova, M. Abdullayeva, and D. Dhabliya, "Leveraging Deep Learning for STEM Education in Higher Secondary Grades," in *Proc. World Skills Conf. on Universal Data Analytics and Sciences (WorldSUAS)*, 2025, pp. 1–6.
- [14] O. David, "Natural Language Processing for Personalized Education in Low-Resource Environments," 2025.
- [15] R. R. Kannan, R. Rajalakshmi, and L. Kumar, "IndicBERT Based Approach for Sentiment Analysis on Code-Mixed Tamil Tweets," in *Proc. FIRE (Working Notes)*, pp. 729–736, 2021.
- [16] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-Efficient Transfer Learning for NLP," in *Proc. Int. Conf. on Machine Learning (ICML)*, 2019, pp. 2790–2799.
- [17] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.
- [18] X. L. Li and P. Liang, "Prefix-Tuning: Optimizing Continuous Prompts for Generation," *arXiv preprint arXiv:2101.00190*, 2021.